$\Omega(n \log n)$
$\Theta(n \log n)$
$\mathcal{O}(n \log n)$

Greedy Method

- **Properties :**
  - Huffman Coding
  - Activity Selection Problem
  - Fractional Knapsack

Activity Selection Problem

- Sort activities by finish time . (Ascending order)
- Select the first activity.
- If Current Start time $\geq$ Previous Finish time, then select.
  $S \geq F \rightarrow$ select.
- Repeat for all activities.

Example:
#

	A_1	A_2	A_3	A_4	A_5	A_6
S	0	3	1	5	5	8
F	6	4	2	9	7	9

After sorting,

	✓	✓	×	✓	×	✓
	A_3	A_2	A_1	A_5	A_4	A_6
S	1	3	0	5	5	8
F	2	4	6	7	9	9

$\therefore$ **Selected Activities :** $A_3 \rightarrow A_2 \rightarrow A_5 \rightarrow A_6$
$\ldots$ **Total Activities = 4**

$\Omega(n \log n)$
$\Theta(n \log n)$
$\mathcal{O}(n \log n)$

Fractional knapsack

- Find P/w for each subject.
- Sort objects in descending order of P/w.
- Pick objects one by one from highest P/w.
- If full object fits, take it completely.
- If not, take the fraction that fits in remaining capacity.
- Add all profits to get maximum profit.

Example:
#

object	1	2	3	4	5	6	7
profit (P)	12	5	16	7	9	11	6
weight (w)	3	1	4	2	9	4	3

(capacity of knapsack) , m = 15 kg

object	1	2	3	4	5	6	7
profit (P)	12	5	16	7	9	11	6
weight (w)	3	1	4	2	9	4	3
P/w	**4**	**5**	**4**	**3.5**	**1**	**2.75**	**2**

object	profit	weight	Remaining Weight
2	5	1	$15 - 1 = 14$
1	12	3	$14 - 3 = 11$
3	16	4	$11 - 4 = 7$
4	7	2	$7 - 2 = 5$
6	11	4	$5 - 4 = 1$
7	2	1	$1 - 1 = 0$
	Total = 53		

. . . **Maximum Profit = 53**

Dynamic Programming

- Create DP table $V[i, w]$: $i =$ item , $w =$ weight
- Initialize first row and first column = 0
- If $w_i > w \rightarrow$ copy upper value
- If $w \geq w_i \rightarrow V[i, w] =$ max $(V[i-1, w], P_i + V[i-1, w-w_i])$
- Fill the table row by row.
- Last cell gives maximum profit.

Example:
#

Item (i)	Profit	Weight (w_i)
1	1	2
2	2	3
3	5	4
4	6	5

$m = 8 \rightarrow$ column $(0 - 8)$
$n = 4 \rightarrow$ row $(0 - 4)$

		Weight								
		0	1	2	3	4	5	6	7	8
Item	0	0	0	0	0	0	0	0	0	0
	1	0	0	1	1	1	1	1	1	1
	2	0	0	1	2	2	3	3	3	3
	3	0	0	1	2	5	5	6	7	7
	4	0	0	1	2	5	6	6	7	**(8)**

$$
\begin{aligned}
&V[4,8] \\
&= \text{max}(V[3,8], 6 + V[3,3]) \\
&= \text{max}(7, 6+2) \\
&= \text{max}(7, 8)
\end{aligned}
$$

$\therefore V[4, 8] = 8$
$\ldots$ **Maximum profit = 8**

$\Omega(n^2)$
$\Theta(n^2)$
$\mathcal{O}(n^2)$

Rod cutting

- Initialize first row and first column = 0
- If row $>$ column $\rightarrow$ copy upper value
- If row $\leq$ column $\rightarrow$ max (exclude, include)
  exclude $=$ above value
  include $=$ profit of current cut length $+$ profit of remaining length (column $-$ row)
- Fill the table row by row
- Last cell gives maximum profit.

Example:
#

Length	1	2	3	4
Profit	2	3	6	9

Total length = 5

Total length

Cut length	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	2	4	6	8	10
2	0	2	4	6	8	10
3	0	2	4	6	8	10
4	0	2	4	6	9	**(11)**

$\max(10, 9+2)$
$= \max(10, 11)$

$\therefore$ **Cut = 4 + 1**
$\therefore$ **Maximum profit = 9 + 2**
= 11

$\Omega(mn)$
$\Theta(mn)$
$\mathcal{O}(mn)$

Longest Common Sequence (LCS)

- Inialize first row and first column = 0
- If characters match $\rightarrow$ Diagonal + 1
- If characters do not match $\rightarrow$ max(top, left)
- Fill the table row by row.
- Last cell value gives the LCS length.
- Backtrack from the last cell to find the LCS string.

Example:
$A =$ STONE
$B =$ LONGEST

A\B	0	L	O	N	G	E	S	T
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
S	**0**	**0**	**0**	**0**	**0**	**0**	**1**	**1**
T	**0**	**0**	**0**	**0**	**0**	**0**	**1**	**2**
O	**0**	**0**	**1**	**1**	**1**	**1**	**1**	**2**
N	**0**	**0**	**1**	**2**	**2**	**2**	**2**	**2**
E	**0**	**0**	**1**	**2**	**2**	**3**	**3**	**3**

Legend:

↗ diagonal ↗ = characters match $\rightarrow$ diag + 1

↑ up ↑ = take max from above cell

← left ← = take max from left cell

▢ box = backtrack path $\rightarrow$ LCS string

LCS length = 3

LCS string = ONE